# Student Ranking System Using BST

## Complete C++ Backend Architecture & Source Code

Data Structures College Project | C++14/C++17 | Winsock REST API

## 1. Project Overview & Architecture

This document contains the complete, production-ready C++ backend for the 'Student Ranking System Using BST' college Data Structures project. The backend is designed as an autonomous high-performance HTTP REST server that executes all Binary Search Tree algorithms in memory and connects to the modern React + Vite frontend.

* **Binary Search Tree Core:** Organizes student records (roll number, name, marks) keyed by marks.
* **O(log N) Search & Insert:** Binary search logic ensures average-case logarithmic operations.
* **Three-Case Deletion:** Implements standard BST deletion for leaf, single-child, and two-children nodes with In-order Successor.
* **O(N) Leaderboard Generation:** Reverse In-Order Traversal (Right -> Root -> Left) generates ranked standings in descending order without external sorting.
* **Dynamic Tree Serialization:** Recursively serializes the pointer hierarchy into JSON for the frontend SVG tree visualizer.
* **Built-in Winsock HTTP Engine:** Standalone HTTP REST server running on port 8080 with full CORS support.

## 2. BST Time & Space Complexity Matrix

| Operation | Average Time | Worst Case (Skewed) | Space Complexity |
|---|---|---|---|
| `Search Student (by Roll/Marks)` | `O(log N)` | `O(N)` | `O(1) auxiliary` |
| `Insert Student Node` | `O(log N)` | `O(N)` | `O(H) recursion stack` |
| `Delete Student Node` | `O(log N)` | `O(N)` | `O(H) recursion stack` |
| `Leaderboard (Reverse In-Order)` | `O(N)` | `O(N)` | `O(N) output buffer` |
| `Tree JSON Serialization` | `O(N)` | `O(N)` | `O(N) JSON payload` |

## 3. C++ HTTP API Endpoint Specification

| Method | Endpoint Route | Description & BST Action |
|---|---|---|
| GET | `/api/health` | Healthcheck and engine status confirmation |
| GET | `/api/students` | Retrieves in-order student list from BST |
| GET | `/api/leaderboard` | Reverse in-order traversal returning rankings sorted by marks |
| GET | `/api/bst/tree` | Hierarchical recursive JSON tree structure for SVG visualizer |
| GET | `/api/stats` | Computes total nodes, class average, topper, and tree height |
| GET | `/api/students/search?q=...` | Searches BST by roll number or name substring |
| POST | `/api/students` | Inserts a new student node into the BST ordered by marks |
| DELETE | `/api/students/:rollNo` | Deletes student node and restructures BST pointers |

## 4. Complete C++ Backend Source Code (backend/server.cpp)

*Compiled with: g++ -O2 -std=c++14 backend/server.cpp -lws2_32 -o backend/server.exe*

```cpp
/**
 * ============================================================================
 * STUDENT RANKING SYSTEM USING BST - C++ HTTP BACKEND
 * ============================================================================
 *
 * College Data Structures Project Backend written purely in C++.
 * Implements:
 *   - Binary Search Tree (BST) with Student struct
 *   - Insertion by marks: O(log N) average
 *   - Deletion with 3 cases (Leaf, 1 Child, 2 Children with In-order Successor)
 *   - In-order and Reverse In-order traversals for ascending & descending ranks
 *   - Dynamic JSON serialization for BST SVG visualizer
 *   - High-performance Winsock HTTP REST Server on port 8080
 *   - Full CORS support for browser frontend
 * ============================================================================
 */

#define WIN32_LEAN_AND_MEAN
#include <winsock2.h>
#include <ws2tcpip.h>
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>
#include <cmath>
#include <cstring>
#include <cctype>

#pragma comment(lib, "ws2_32.lib")

// Student Entity
struct Student {
    int rollNo;
    std::string name;
    double marks;
    int rank;
};

// BST Node
struct BSTNode {
    Student student;
    BSTNode* left;
    BSTNode* right;

    BSTNode(Student s) : student(s), left(nullptr), right(nullptr) {}
};

// Binary Search Tree Engine
class StudentBST {
private:
    BSTNode* root;
    int totalCount;

    // Helper: Insert by marks (standard BST)
    BSTNode* insertHelper(BSTNode* node, const Student& s) {
        if (node == nullptr) {
            totalCount++;
            return new BSTNode(s);
        }
        if (s.marks < node->student.marks) {
            node->left = insertHelper(node->left, s);
        } else {
            node->right = insertHelper(node->right, s);
        }
        return node;
```

```
    }

    // Helper: Find minimum node (In-Order Successor)
    BSTNode* findMin(BSTNode* node) {
        while (node && node->left != nullptr) {
            node = node->left;
        }
        return node;
    }

    // Helper: Delete by roll number
    BSTNode* deleteHelper(BSTNode* node, int rollNo, bool& deleted) {
        if (node == nullptr) return nullptr;

        if (rollNo == node->student.rollNo) {
            deleted = true;
            totalCount--;

            // Case 1: Leaf Node (0 children)
            if (node->left == nullptr && node->right == nullptr) {
                delete node;
                return nullptr;
            }
            // Case 2: One Child (Right only)
            if (node->left == nullptr) {
                BSTNode* temp = node->right;
                delete node;
                return temp;
            }
            // Case 2: One Child (Left only)
            if (node->right == nullptr) {
                BSTNode* temp = node->left;
                delete node;
                return temp;
            }
            // Case 3: Two Children (Replace with In-order Successor)
            BSTNode* successor = findMin(node->right);
            node->student = successor->student;
            bool dummy = false;
            node->right = deleteHelper(node->right, successor->student.rollNo, dummy);
            totalCount++; // Balance decrement
            return node;
        }

        node->left = deleteHelper(node->left, rollNo, deleted);
        if (!deleted) {
            node->right = deleteHelper(node->right, rollNo, deleted);
        }
        return node;
    }

    // Traversal: Reverse In-Order (Right -> Root -> Left) for Descending Rank
    void reverseInOrder(BSTNode* node, std::vector<Student>& list) const {
        if (node == nullptr) return;
        reverseInOrder(node->right, list);
        list.push_back(node->student);
        reverseInOrder(node->left, list);
    }

    // Traversal: In-Order (Left -> Root -> Right)
    void inOrder(BSTNode* node, std::vector<Student>& list) const {
        if (node == nullptr) return;
        inOrder(node->left, list);
        list.push_back(node->student);
        inOrder(node->right, list);
    }

    // Search helper by Roll Number
    BSTNode* searchRoll(BSTNode* node, int rollNo) const {
        if (node == nullptr) return nullptr;
```

```
        if (node->student.rollNo == rollNo) return node;
        BSTNode* res = searchRoll(node->left, rollNo);
        if (res) return res;
        return searchRoll(node->right, rollNo);
    }

    // Search helper by Name substring
    void searchName(BSTNode* node, const std::string& query, std::vector<Student>& results) const {
        if (node == nullptr) return;
        searchName(node->left, query, results);

        std::string sName = node->student.name;
        std::string q = query;
        std::transform(sName.begin(), sName.end(), sName.begin(), ::tolower);
        std::transform(q.begin(), q.end(), q.begin(), ::tolower);

        if (sName.find(q) != std::string::npos) {
            results.push_back(node->student);
        }
        searchName(node->right, query, results);
    }

    // Height calculation
    int getHeight(BSTNode* node) const {
        if (node == nullptr) return 0;
        return 1 + std::max(getHeight(node->left), getHeight(node->right));
    }

    // Recursive JSON serialization for BST Visualizer
    std::string serializeNodeJSON(BSTNode* node) const {
        if (node == nullptr) return "null";
        std::stringstream ss;
        ss << "{"
           << "\"rollNo\":" << node->student.rollNo << ","
           << "\"name\":\"" << escapeJSON(node->student.name) << "\","
           << "\"marks\":" << node->student.marks << ","
           << "\"rank\":" << node->student.rank << ","
           << "\"left\":" << serializeNodeJSON(node->left) << ","
           << "\"right\":" << serializeNodeJSON(node->right)
           << "}";
        return ss.str();
    }

    static std::string escapeJSON(const std::string& input) {
        std::string output;
        for (char c : input) {
            if (c == '"') output += "\\\"";
            else if (c == '\\') output += "\\\\";
            else if (c == '\n') output += "\\n";
            else if (c == '\r') output += "\\r";
            else if (c == '\t') output += "\\t";
            else output += c;
        }
        return output;
    }

public:
    StudentBST() : root(nullptr), totalCount(0) {}

    bool insert(int rollNo, const std::string& name, double marks) {
        if (searchByRoll(rollNo) != nullptr) {
            return false; // Duplicate roll number rejected
        }
        Student s = { rollNo, name, marks, 1 };
        root = insertHelper(root, s);
        updateRanks();
        return true;
    }

    bool deleteStudent(int rollNo) {
```

```
        bool deleted = false;
        root = deleteHelper(root, rollNo, deleted);
        if (deleted) updateRanks();
        return deleted;
    }

    BSTNode* searchByRoll(int rollNo) const {
        return searchRoll(root, rollNo);
    }

    std::vector<Student> searchStudents(const std::string& query) const {
        std::vector<Student> results;
        if (query.empty()) return results;

        // Check if query is numeric roll number
        char* endPtr = nullptr;
        long roll = std::strtol(query.c_str(), &endPtr, 10);
        if (endPtr != query.c_str() && *endPtr == '\0') {
            BSTNode* found = searchByRoll(static_cast<int>(roll));
            if (found) {
                results.push_back(found->student);
                return results;
            }
        }

        // Substring search on student names
        searchName(root, query, results);
        return results;
    }

    std::vector<Student> getLeaderboard() const {
        std::vector<Student> list;
        reverseInOrder(root, list);
        return list;
    }

    std::vector<Student> getAllStudents() const {
        std::vector<Student> list;
        inOrder(root, list);
        return list;
    }

    void updateRanks() {
        std::vector<Student> descList;
        reverseInOrder(root, descList);
        int currentRank = 1;
        for (size_t i = 0; i < descList.size(); i++) {
            if (i > 0 && descList[i].marks < descList[i - 1].marks) {
                currentRank = static_cast<int>(i + 1);
            }
            BSTNode* n = searchByRoll(descList[i].rollNo);
            if (n) n->student.rank = currentRank;
        }
    }

    int getHeight() const {
        return getHeight(root);
    }

    int size() const {
        return totalCount;
    }

    std::string toJSONTree() const {
        std::stringstream ss;
        ss << "{"
           << "\"root\":" << serializeNodeJSON(root) << ","
           << "\"totalNodes\":" << totalCount << ","
           << "\"treeHeight\":" << getHeight() << ","
           << "\"keyProperty\":\"marks\""
```

```
            << "}";
        return ss.str();
    }

    std::string getStatsJSON() const {
        std::vector<Student> list = getLeaderboard();
        double sum = 0.0;
        for (const auto& s : list) sum += s.marks;
        double avg = list.empty() ? 0.0 : (sum / list.size());

        std::stringstream ss;
        ss << "{"
           << "\"totalStudents\":" << totalCount << ","
           << "\"averageMarks\":" << avg << ","
           << "\"treeHeight\":" << getHeight() << ",";
        if (list.empty()) {
            ss << "\"topper\":null";
        } else {
            ss << "\"topper\":{"
               << "\"rollNo\":" << list[0].rollNo << ","
               << "\"name\":\"" << escapeJSON(list[0].name) << "\","
               << "\"marks\":" << list[0].marks << ","
               << "\"rank\":1"
               << "}";
        }
        ss << "}";
        return ss.str();
    }
};

// Global BST Instance
StudentBST g_tree;

// JSON Helpers
std::string studentsToJSON(const std::vector<Student>& list) {
    std::stringstream ss;
    ss << "[";
    for (size_t i = 0; i < list.size(); i++) {
        ss << "{"
           << "\"rollNo\":" << list[i].rollNo << ","
           << "\"name\":\"" << list[i].name << "\","
           << "\"marks\":" << list[i].marks << ","
           << "\"rank\":" << list[i].rank
           << "}";
        if (i + 1 < list.size()) ss << ",";
    }
    ss << "]";
    return ss.str();
}

// Robust JSON field parser
bool parseStudentJSON(const std::string& body, int& rollNo, std::string& name, double& marks) {
    auto extractValue = [&](const std::string& key) -> std::string {
        std::string pattern = "\"" + key + "\"";
        size_t pos = body.find(pattern);
        if (pos == std::string::npos) return "";

        size_t colon = body.find(':', pos + pattern.length());
        if (colon == std::string::npos) return "";

        size_t start = colon + 1;
        while (start < body.length() && (body[start] == ' ' || body[start] == '\t' || body[start] == '\r' || body[start] ==
            start++;
        }
        if (start >= body.length()) return "";

        if (body[start] == '"') {
            size_t end = body.find('"', start + 1);
            if (end == std::string::npos) return "";
            return body.substr(start + 1, end - start - 1);
```

```
        } else {
            size_t end = start;
            while (end < body.length() && body[end] != ',' && body[end] != '}' && body[end] != ' ' && body[end] != '\r' &&
                end++;
            }
            return body.substr(start, end - start);
        }
    };

    std::string rStr = extractValue("rollNo");
    std::string nStr = extractValue("name");
    std::string mStr = extractValue("marks");

    if (rStr.empty() || nStr.empty() || mStr.empty()) return false;

    try {
        rollNo = std::stoi(rStr);
        name = nStr;
        marks = std::stod(mStr);
        return true;
    } catch (...) {
        return false;
    }
}

// URL query parameter decoder
std::string urlDecode(const std::string& str) {
    std::string res;
    for (size_t i = 0; i < str.length(); ++i) {
        if (str[i] == '+') {
            res += ' ';
        } else if (str[i] == '%' && i + 2 < str.length()) {
            std::string hex = str.substr(i + 1, 2);
            char c = static_cast<char>(std::strtol(hex.c_str(), nullptr, 16));
            res += c;
            i += 2;
        } else {
            res += str[i];
        }
    }
    return res;
}

// Send HTTP response
void sendHTTPResponse(SOCKET client, int statusCode, const std::string& statusText, const std::string& body, const std::str
    std::stringstream response;
    response << "HTTP/1.1 " << statusCode << " " << statusText << "\r\n";
    response << "Content-Type: " << contentType << "\r\n";
    response << "Content-Length: " << body.length() << "\r\n";
    response << "Access-Control-Allow-Origin: *\r\n";
    response << "Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS\r\n";
    response << "Access-Control-Allow-Headers: Content-Type, Accept, Authorization\r\n";
    response << "Connection: close\r\n\r\n";
    response << body;

    std::string respStr = response.str();
    send(client, respStr.c_str(), static_cast<int>(respStr.length()), 0);
}

// Handle Single HTTP Request with Full Body Reception
void handleClient(SOCKET client) {
    std::string rawRequest = "";
    char buffer[4096];

    // Read headers first
    size_t headerEnd = std::string::npos;
    size_t delimiterLen = 4;

    while (headerEnd == std::string::npos) {
        int bytesRead = recv(client, buffer, sizeof(buffer) - 1, 0);
```

```
        if (bytesRead <= 0) {
            closesocket(client);
            return;
        }
        buffer[bytesRead] = '\0';
        rawRequest.append(buffer, bytesRead);

        headerEnd = rawRequest.find("\r\n\r\n");
        if (headerEnd != std::string::npos) {
            delimiterLen = 4;
        } else {
            headerEnd = rawRequest.find("\n\n");
            if (headerEnd != std::string::npos) {
                delimiterLen = 2;
            }
        }
    }

    // Determine Content-Length if present
    int contentLength = 0;
    std::string lowerReq = rawRequest;
    std::transform(lowerReq.begin(), lowerReq.end(), lowerReq.begin(), ::tolower);
    size_t clPos = lowerReq.find("content-length:");
    if (clPos != std::string::npos) {
        size_t lineEnd = rawRequest.find("\n", clPos);
        if (lineEnd != std::string::npos) {
            std::string clStr = rawRequest.substr(clPos + 15, lineEnd - (clPos + 15));
            try {
                contentLength = std::stoi(clStr);
            } catch (...) {
                contentLength = 0;
            }
        }
    }

    // Read remaining body if needed
    size_t bodyStart = headerEnd + delimiterLen;
    while (static_cast<int>(rawRequest.length() - bodyStart) < contentLength) {
        int bytesRead = recv(client, buffer, sizeof(buffer) - 1, 0);
        if (bytesRead <= 0) break;
        rawRequest.append(buffer, bytesRead);
    }

    std::string headersPart = rawRequest.substr(0, headerEnd);
    std::string body = (bodyStart < rawRequest.length()) ? rawRequest.substr(bodyStart) : "";

    // Parse HTTP request line
    std::istringstream reqStream(headersPart);
    std::string method, path, httpVersion;
    reqStream >> method >> path >> httpVersion;

    std::cout << "[HTTP " << method << "] " << path << std::endl;

    // 1. CORS Preflight
    if (method == "OPTIONS") {
        sendHTTPResponse(client, 200, "OK", "", "text/plain");
        closesocket(client);
        return;
    }

    // 2. GET /api/health
    if (method == "GET" && path == "/api/health") {
        sendHTTPResponse(client, 200, "OK", "{\"status\":\"online\",\"engine\":\"C++ BST Engine (Winsock)\",\"totalStudents
        closesocket(client);
        return;
    }

    // 3. GET /api/students
    if (method == "GET" && (path == "/api/students" || path == "/api/students/")) {
        std::string json = "{\"students\":" + studentsToJSON(g_tree.getAllStudents()) + "}";
```

```
        sendHTTPResponse(client, 200, "OK", json);
        closesocket(client);
        return;
    }

    // 4. GET /api/leaderboard
    if (method == "GET" && (path == "/api/leaderboard" || path == "/api/leaderboard/")) {
        std::string json = "{\"leaderboard\":" + studentsToJSON(g_tree.getLeaderboard()) + "}";
        sendHTTPResponse(client, 200, "OK", json);
        closesocket(client);
        return;
    }

    // 5. GET /api/bst/tree
    if (method == "GET" && (path == "/api/bst/tree" || path == "/api/bst/tree/")) {
        std::string json = "{\"tree\":" + g_tree.toJSONTree() + "}";
        sendHTTPResponse(client, 200, "OK", json);
        closesocket(client);
        return;
    }

    // 6. GET /api/stats
    if (method == "GET" && (path == "/api/stats" || path == "/api/stats/")) {
        std::string json = "{\"stats\":" + g_tree.getStatsJSON() + "}";
        sendHTTPResponse(client, 200, "OK", json);
        closesocket(client);
        return;
    }

    // 7. GET /api/students/search?q=...
    if (method == "GET" && path.rfind("/api/students/search", 0) == 0) {
        size_t qPos = path.find("q=");
        std::string query = "";
        if (qPos != std::string::npos) {
            size_t endPos = path.find('&', qPos);
            query = path.substr(qPos + 2, (endPos == std::string::npos) ? std::string::npos : endPos - (qPos + 2));
            query = urlDecode(query);
        }
        std::vector<Student> matches = g_tree.searchStudents(query);
        std::string json = "{\"results\":" + studentsToJSON(matches) + "}";
        sendHTTPResponse(client, 200, "OK", json);
        closesocket(client);
        return;
    }

    // 8. POST /api/students (Insert Node into BST)
    if (method == "POST" && (path == "/api/students" || path == "/api/students/")) {
        int rollNo = 0;
        std::string name;
        double marks = 0.0;
        if (!parseStudentJSON(body, rollNo, name, marks)) {
            sendHTTPResponse(client, 400, "Bad Request", "{\"error\":\"Invalid student JSON payload\"}");
            closesocket(client);
            return;
        }

        if (rollNo <= 0 || marks < 0 || marks > 100 || name.empty()) {
            sendHTTPResponse(client, 400, "Bad Request", "{\"error\":\"Validation failed: Check rollNo (>0), name, and mark
            closesocket(client);
            return;
        }

        bool success = g_tree.insert(rollNo, name, marks);
        if (!success) {
            sendHTTPResponse(client, 409, "Conflict", "{\"error\":\"Student with Roll No #" + std::to_string(rollNo) + " al
            closesocket(client);
            return;
        }

        BSTNode* inserted = g_tree.searchByRoll(rollNo);
```

```
        std::stringstream resp;
        resp << "{\"success\":true,\"student\":{"
             << "\"rollNo\":" << rollNo << ","
             << "\"name\":\"" << name << "\","
             << "\"marks\":" << marks << ","
             << "\"rank\":" << (inserted ? inserted->student.rank : 1)
             << "}}";
        sendHTTPResponse(client, 201, "Created", resp.str());
        closesocket(client);
        return;
    }

    // 9. DELETE /api/students/:rollNo (Delete Node from BST)
    if (method == "DELETE" && path.rfind("/api/students/", 0) == 0) {
        std::string rollStr = path.substr(std::string("/api/students/").length());
        size_t queryStart = rollStr.find('?');
        if (queryStart != std::string::npos) rollStr = rollStr.substr(0, queryStart);

        try {
            int rollNo = std::stoi(rollStr);
            bool deleted = g_tree.deleteStudent(rollNo);
            if (deleted) {
                sendHTTPResponse(client, 200, "OK", "{\"success\":true,\"message\":\"Student #" + std::to_string(rollNo) +
            } else {
                sendHTTPResponse(client, 404, "Not Found", "{\"error\":\"Student #" + std::to_string(rollNo) + " not found
            }
        } catch (...) {
            sendHTTPResponse(client, 400, "Bad Request", "{\"error\":\"Invalid roll number parameter\"}");
        }
        closesocket(client);
        return;
    }

    // Default 404
    sendHTTPResponse(client, 404, "Not Found", "{\"error\":\"Route not found in C++ BST Backend\"}");
    closesocket(client);
}

int main() {
    std::cout << "============================================================\n";
    std::cout << " Student Ranking System Using BST - C++ HTTP Server\n";
    std::cout << "============================================================\n";

    // Pre-populate C++ BST with initial sample college students
    g_tree.insert(101, "Aarav Sharma", 95.0);
    g_tree.insert(104, "Diya Patel", 92.0);
    g_tree.insert(106, "Rohan Kulkarni", 87.0);
    g_tree.insert(102, "Ananya Iyer", 81.0);
    g_tree.insert(105, "Vikram Malhotra", 74.0);
    g_tree.insert(103, "Sneha Reddy", 68.0);
    g_tree.insert(107, "Kabir Mehta", 59.0);

    std::cout << "[BST Initialized] 7 initial students seeded into Binary Search Tree.\n";
    std::cout << "[BST Height] " << g_tree.getHeight() << " levels. Initial Root: Roll #101\n";

    // Initialize Winsock
    WSADATA wsaData;
    int wsaRes = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (wsaRes != 0) {
        std::cerr << "WSAStartup failed with error: " << wsaRes << std::endl;
        return 1;
    }

    SOCKET listenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listenSocket == INVALID_SOCKET) {
        std::cerr << "socket creation failed: " << WSAGetLastError() << std::endl;
        WSACleanup();
        return 1;
    }

```

```
    int opt = 1;
    setsockopt(listenSocket, SOL_SOCKET, SO_REUSEADDR, (const char*)&opt, sizeof(opt));

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(8080);

    if (bind(listenSocket, (sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        std::cerr << "Bind failed with error: " << WSAGetLastError() << std::endl;
        closesocket(listenSocket);
        WSACleanup();
        return 1;
    }

    if (listen(listenSocket, SOMAXCONN) == SOCKET_ERROR) {
        std::cerr << "Listen failed with error: " << WSAGetLastError() << std::endl;
        closesocket(listenSocket);
        WSACleanup();
        return 1;
    }

    std::cout << "\n>>> C++ BST Backend Server is LIVE and listening at http://localhost:8080 <<<\n";
    std::cout << "Ready to accept HTTP requests from React frontend at http://localhost:5173\n";
    std::cout << "Endpoints active:\n";
    std::cout << "  - GET    /api/health\n";
    std::cout << "  - GET    /api/students\n";
    std::cout << "  - GET    /api/leaderboard\n";
    std::cout << "  - GET    /api/bst/tree\n";
    std::cout << "  - GET    /api/stats\n";
    std::cout << "  - GET    /api/students/search?q=...\n";
    std::cout << "  - POST   /api/students\n";
    std::cout << "  - DELETE /api/students/:rollNo\n\n";

    // Accept loop
    while (true) {
        SOCKET clientSocket = accept(listenSocket, nullptr, nullptr);
        if (clientSocket == INVALID_SOCKET) {
            std::cerr << "Accept failed with error: " << WSAGetLastError() << std::endl;
            continue;
        }
        handleClient(clientSocket);
    }

    closesocket(listenSocket);
    WSACleanup();
    return 0;
}
```